Chapter - 1
Learning From Data

Aviral Janveja

1 Introduction

Let us look at the fundamentals of machine learning, using an example from the domain of finance. Suppose someone applies for a bank loan. Now, the bank must decide whether granting the loan is a good idea or not, since their goal is to maximize profit while minimizing risk. Naturally, the bank has no magic formula to predict whether a customer will make timely repayments or not. However, it is indeed possible to look at similar customers from the past and examine how their repayment behavior turned out to be.

The **key idea** here, is to use past data points to uncover an underlying pattern that connects customer information with their repayment behavior. So that, we can use it to predict the creditworthiness of fresh applicants. Using this example, let us now outline the main elements of the machine learning process :

1.1 The Underlying Pattern

As discussed above, in machine learning, the idea is to uncover the underlying pattern from a given set of data points. In the bank loan example, this underlying pattern is the relation that connects customer information with their repayment behavior. The pattern is **essentially unknown** to us, but we do get glimpses of it through the available data points.

Target Function

In this section, we will describe this underlying pattern mathematically. But first, let us first consider a simpler scenario. Suppose the bank employs the following rule for loan approval :

> Approve every applicant whose annual income exceeds Rs. 7 lakh, otherwise reject the application.

This is a simple rule with no uncertainty. Every input produces exactly one correct output. Such a relation can be easily represented via a regular mathematical function, formally called the **target function** :

$$f(\mathbf{x})$$

For now, let us consider income as the only input for simplicity. So, if x = Rs. 9 lakhs per annum, then $f(9) = +1$, indicating that the loan shall be approved, as per the above rule.

Target Distribution

However, we know that real world decision making is rarely this simple. Two applicants may have identical ages, salaries and employment histories, yet one eventually repays the loan while the other defaults. This can happen due to factors not captured in our data, such as job loss, unexpected medical expenses or simply the inherent uncertainty in human life. This leads us to an important observation :

> Most real world scenarios that we will deal with in machine learning are probabilistic in nature.

Hence, instead of fixed output functions, we get the probabilities of different possible outputs, for a given input. For example :

$$P(y = +1 \mid x = 9) = 0.80$$
$$P(y = -1 \mid x = 9) = 0.20$$

Meaning that an applicant with a salary of Rs. 9 lakhs per annum, has an 80% probability of repaying the loan and a 20% probability of defaulting.

Therefore, instead of assuming a fixed output for every input, we describe the underlying pattern in terms of probabilities of different possible outputs, for a given input. Mathematically, this is represented by a conditional probability distribution, formally called the **target distribution** :

$$P(y \mid \mathbf{x})$$

It gives us the probabilities of all possible outputs y, for a given input $\mathbf{x}$, as shown in the above example.

It is also important to **note** that the target function is simply a special case of the more general concept of target distribution. For example, when the probability of a certain output is 1 and thereby 0 for all others, the target distribution collapses to a target function :

$$P(y \mid \mathbf{x}) = \begin{cases} 1 & \text{if } y = f(\mathbf{x}) \\ 0 & \text{if } y \neq f(\mathbf{x}) \end{cases}$$

Input Distribution

So, the target distribution is the relation that we are trying to learn. However, there is another important probability distribution that underlies the inputs, called the **input distribution** :

$$P(\mathbf{x})$$

It captures the probability of picking a given customer profile versus the other.

For example, if we consider annual income as our only input, the input distribution quantifies the likelihood of various income brackets in the population.

It is a very natural addition. Think about it. It is already implicit in the data. Let us consider a simple example, a pot of 100 marbles, 70 red and 30 green. Even here, there is already a probability of picking one color marble versus the other.

For the pot example, this means sampling with replacement, or equivalently treating each draw as an independent draw from an identical 70/30 distribution. This is called independent and identically distributed (**I.I.D**) sampling in statistics and machine learning.

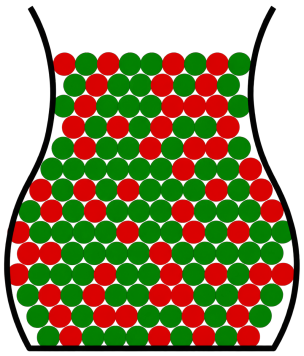


Figure 1: Pot of Marbles

Joint Distribution

Together, they are referred to as the **joint distribution**, which describes the data generation process in its entirety, including both the input and output :

$$P(\mathbf{x})\ P(y \mid \mathbf{x}) \implies P(\mathbf{x}, y)$$

1.2 The Data Set

The data set is simply a collection of data points generated by the unknown joint probability distribution, as described above. In the loan approval example, each data point corresponds to a past customer and collecting N such data points gives us the required data set :

$$\mathbf{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), (\mathbf{x}_3, y_3)...(\mathbf{x}_N, y_N)\}$$

Where $\mathbf{x}_n$ is the application information of the nth customer, represented by an array of numbers as shown below :

$$\mathbf{x}_n = \begin{bmatrix} x_1 \\ x_2 \\ . \\ . \\ x_d \end{bmatrix}$$

Here x_1, x_2 and so on upto x_d are data attributes of a customer like age, annual income, years in residence, outstanding debt and so on. While the corresponding output :

$$y_n \in \{+1, -1\}$$

Represents whether that customer eventually turned out to be creditworthy or not. Now, the data set does not reveal the target distribution completely, but it

provides us with examples using which we can attempt to approximate it. In other words, the data set acts as our limited window into the underlying pattern.

2 Our First Learning Model : The Perceptron

So far, we have examined the two fundamental ingredients of every machine learning process, the **underlying pattern** and the **data set**. The next question is :

> How do we use the data set to learn something useful about the target distribution ?

To answer this question, we introduce perceptron, a simple linear model. Let us suppose that for a particular customer :

$$P(y = +1 \mid \mathbf{x}_n) = 0.80$$
$$P(y = -1 \mid \mathbf{x}_n) = 0.20$$

The perceptron does not attempt to estimate these probabilities. Instead, it only predicts the final binary output. In this case, it would simply predict :

$$y = +1$$

Although, this is only a simplified, hard approximation of the underlying probability distribution, it provides an excellent starting point for understanding how machine learning models work. More advanced learning models introduced later will attempt to capture the probability distribution more explicitly.

2.1 Hypothesis Function

The perceptron works by assigning a numerical weight to each input data attribute, such as age, income and so on. The weighted attributes are then summed together to produce a single score :

$$w_1 x_1 + w_2 x_2 + ... + w_d x_d$$

This weighted sum can be interpreted as a credit score assigned by the perceptron. The idea is pretty simple, if the credit score is greater than a certain threshold, the loan is approved, otherwise it is denied :

$$w_1 x_1 + w_2 x_2 + ... + w_d x_d > \text{threshold} \quad \text{(Approve)}$$
$$w_1 x_1 + w_2 x_2 + ... + w_d x_d \leq \text{threshold} \quad \text{(Deny)}$$

For mathematical convenience, we rewrite the threshold as another weight :

$$w_0 = -\text{threshold}$$

along with an artificial attribute to go along with it :

$$x_0 = 1$$

Hence, the decision rule now becomes :

$$w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d > 0 \quad \text{(Approve)}$$
$$w_0x_0 + w_1x_1 + w_2x_2 + ... + w_dx_d \leq 0 \quad \text{(Deny)}$$

Next, we write the above expression in vector notation to make it clean and concise :

$$w_0x_0 + w_1x_1 + .. + w_dx_d \; = \begin{bmatrix} w_0 & w_1 & . & . & w_d \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ . \\ . \\ x_d \end{bmatrix} \; = \; \mathbf{w^T x}$$

Since, the above formula outputs a real number that could be greater than, equal to or less than zero whereas our desired output is binary, we pass this value through the **sign** function, which outputs $+1$ if the value is greater than zero and -1 otherwise, hence giving us the final perceptron hypothesis function :

$$h(\mathbf{x}) = \text{sign}(\mathbf{w^T x})$$

Hypothesis Set

The mathematical form of the hypothesis function is now fixed. The only unknown quantities are the weights contained in the vector $\mathbf{w}$. Weights are real numbers and can therefore take infinitely many different values.

Each different choice of the weight vector thereby produces a different hypothesis. The collection of all such candidate hypotheses is called the hypothesis set, denoted by :

$$\mathbf{H} = \{h_1, h_2, h_3....\}$$

2.2 Perceptron Learning Algorithm

With the hypothesis function fixed, the task now is to determine the values of the weight vector :

$$\mathbf{w}$$

We begin by initializing the weights randomly, usually to zero. The perceptron learning algorithm then iterates through the data set, searching for **errors**. For a single data point $(\mathbf{x}, y)$, the **point-wise error** is denoted by :

$$e(h(\mathbf{x}), y) \implies h(\mathbf{x}_n) \neq y_n \implies \text{sign}(\mathbf{w^T x}_n) \neq y_n$$

That is, when our hypothesis function disagrees with the observed output, on a particular data point, hence making it a misclassified point. Taking average of the point-wise errors over the entire data set, gives us the total **in-sample error** :

$$E_{in}(h) = \frac{1}{N} \sum_{n=1}^{N} e(h(\mathbf{x}_n), y_n)$$

In order to minimize the in-sample error, the learning algorithm updates the weight vector whenever a misclassified point is found, by applying the following **update rule** :

$$\mathbf{w}_{new} = \mathbf{w} + y_n \mathbf{x}_n$$

The learning algorithm continues making such updates to the weight vector, until no misclassified points remain in the data set or another stopping criterion is reached. Thereby arriving at the final set of weights that yield the lowest in-sample error :

$$E_{in} \approx 0$$

And that is it, the final weight vector gives us the **final hypothesis** function :

$$h_f(\mathbf{x})$$

2.3 Update Rule Justification

For now, let us examine how the above update rule actually nudges the perceptron in the right direction at each misclassified point. If a point $(\mathbf{x}_n, y_n)$ is misclassified, it means :

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) \neq y_n$$

Accordingly, the perceptron updates the weight vector as follows :

$$\mathbf{w}_{new} = \mathbf{w} + y_n \mathbf{x}_n$$

Substituting the updated weight vector into the linear score gives :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = (\mathbf{w} + y_n \mathbf{x}_n)^\mathbf{T}\mathbf{x}_n$$

Expanding the right-hand side, we get :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + y_n \mathbf{x}_n^\mathbf{T}\mathbf{x}_n$$

This is equivalent to :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + y_n ||\mathbf{x}_n||^2$$

Now, for a misclassified point, where the observed output is :

$$y_n = +1$$

Whereas our hypothesis function predicted :

$$\text{sign}(\mathbf{w}^\mathbf{T}\mathbf{x}_n) = -1 \implies \mathbf{w}^\mathbf{T}\mathbf{x}_n \leq 0$$

After the update, we get :

$$\mathbf{w}_{new}^\mathbf{T}\mathbf{x}_n = \mathbf{w}^\mathbf{T}\mathbf{x}_n + ||\mathbf{x}_n||^2$$

Since,

$$||\mathbf{x}_n||^2 > 0$$

The update increases the score assigned to that particular customer. This might not lead to an immediate sign correction, but it is clearly pushing it towards the correct side.

Similarly, when :

$$y_n = -1$$

And our hypothesis function predicts :

$$\text{sign}(\mathbf{w^T x}_n) = +1 \implies \mathbf{w^T x}_n > 0$$

The update gives us :

$$\mathbf{w}_{new}^{\mathbf{T}} \mathbf{x}_n = \mathbf{w^T x}_n - ||\mathbf{x}_n||^2$$

Hence, showing that each update to the weight vector nudges the perceptron in the required direction, improving its performance on that particular data-point.

Geometric Interpretation

Let us now examine the perceptron update rule from the lens of vector geometry. The perceptron hypothesis function classifies the data points as follows :

$$h(\mathbf{x}) = \text{sign}(\mathbf{w^T x})$$

The vector multiplication inside the sign function, can be written in the dot product notation as follows :

$$\mathbf{w^T x} = |\mathbf{w}|\ |\mathbf{x}|\ cos\theta$$

Consider an example, where the true output is $y = +1$, but our current hypothesis function misclassifies it as -1, as shown in the image below :

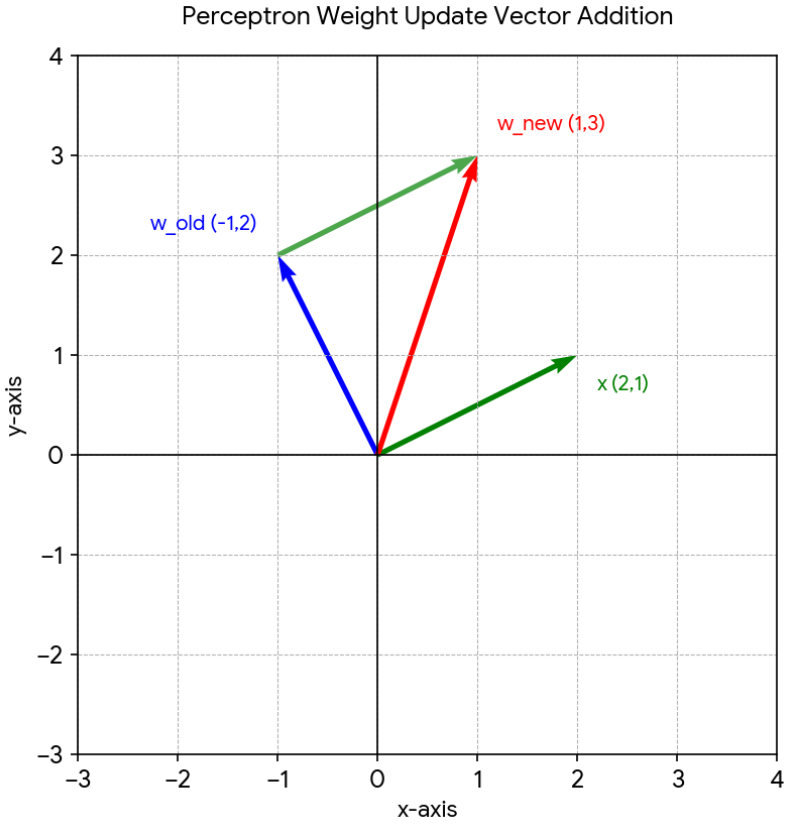


Figure 2: Vector Geometry

Since,

$$\text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}) = -1 \implies \mathbf{w}^{\mathbf{T}}\mathbf{x} \leq 0 \implies |\mathbf{w}|\,|\mathbf{x}|\,cos\theta \leq 0$$

Which makes sense because the angle between $\mathbf{w}_{old}$ and $\mathbf{x}$ is between $90°$ and $180°$ and $\cos\theta$ is less than or equal to zero in this range. Since the above point is misclassified, as per the update rule, we update the weight vector as follows :

$$\mathbf{w}_{new} = \mathbf{w}_{old} + y\mathbf{x} \implies \mathbf{w}_{new} = \mathbf{w}_{old} + \mathbf{x}$$

The effect of this update on the angle between $\mathbf{w}_{new}$ and $\mathbf{x}$ is visible in the above image. The angle is now acute and hence the dot product will be now be positive, since $cos\theta > 0$, when the angle is between $0°$ and $90°$. So now :

$$|\mathbf{w}|\,|\mathbf{x}|\,cos\theta > 0 \implies \text{sign}(\mathbf{w}^{\mathbf{T}}\mathbf{x}) = +1 = y$$

Hence correcting the misclassified point. In general, while a single update might not correct the classification of a data point immediately, as shown in the above example, it clearly moves $\mathbf{w}_{new}$ in the required direction with respect to $\mathbf{x}$, such that the classification eventually becomes correct, possibly after multiple updates.

Convergence Theorem

The convergence theorem for perceptron guarantees that if the data set is **linearly separable**, repeated updates to the weight vector via the update rule will eventually classify all points correctly.

Data set being linearly separable simply means that, when the data points are plotted on a graph, it is possible to draw a straight-line that completely separates them into two distinct classes.

> This theorem is stated here without proof, as the proof is beyond the scope and requirement of this chapter.

3 Hoeffding Inequality

However, remember that the true objective of machine learning is not about performing well on the available data set, but to perform well on future unseen examples generated from the same joint distribution. That is, to have a low **out-of-sample error** :

$$E_{out}(h_f) = \mathbb{E}_{(\mathbf{x},y)}[e(h_f(\mathbf{x}), y)]$$

Which represents the **expected error** over future data points drawn from the same joint distribution. Since, both the target distribution and the input distribution are **unknown**, out-of-sample error cannot be computed directly. Instead, the learning model minimizes the in-sample error in the **hope** that the resulting final hypothesis will also achieve a small out-of-sample error, that is :

$$E_{out} \approx E_{in}$$

Which will be the primary topic for the next chapter, on the **theory of learning**.

4 References

1. CalTech Machine Learning Course. Learning From Data (CS156) : Lecture 1 - The Learning Problem.
2. CalTech Machine Learning Course. Learning From Data (CS156) : Lecture 2 - Is Learning Feasible?
3. CalTech Machine Learning Course. Learning From Data (CS156) : Lecture 4 - Error and Noise.